



Universidad Tecnológica de Calvillo

Frameworks para el desarrollo multiplataforma

Manual básico de DevOps para una aplicación Ionic

Integrantes:

Victor Escalera Solis

Arturo Muñoz Garcia

Ian Daniel Rodriguez

9A ITIID

Jorge de Lara Hernández

27/07/2026

1.1 Descripción del proyecto

Problema que resuelve

En la rutina diaria, los usuarios requieren una forma rápida, ligera y accesible de registrar pensamientos, pendientes e información clave en sus dispositivos móviles sin la complejidad ni la sobrecarga de aplicaciones pesadas que exigen autenticación o conexión constante a internet.

Usuarios a los que está dirigida

La aplicación está dirigida a estudiantes, profesionistas y usuarios generales que necesitan un organizador personal de notas intuitivo, rápido y disponible en múltiple idioma.

Objetivo de la aplicación

Proporcionar una aplicación móvil y web híbrida que permita gestionar notas personales mediante un sistema intuitivo de creación, edición y visualización, incorporando soporte bilingüe (español e inglés) e integración con prácticas de despliegue automatizado.

Funciones principales

- **Visualización e historial de notas:** Listado organizado de notas registradas.
- **Gestión de notas:** Formulario para la creación y edición de registros con validación básica de campos.
- **Internacionalización:** Cambio dinámico de idioma entre español e inglés en toda la interfaz.
- **Persistencia local:** Almacenamiento eficiente de la información mediante arreglos y almacenamiento local.

Tecnologías utilizadas

- **Framework:** Ionic Framework (Angular Standalone Components).
- **Lenguajes:** TypeScript, HTML5, SCSS.
- **Gestión de Estado y Servicios:** RxJS / Angular Services (NotesService, LanguageService).
- **Internacionalización:** Archivos de traducción basados en JSON (assets/i18n/).
- **Control de versiones:** Git y repositorio hospedado en GitHub.

1.2 Instalación y ejecución

Requisitos necesarios

1. **Node.js:** Versión LTS (v18 o superior).
2. **npm:** Gestor de paquetes de Node.
3. **Ionic CLI:** Instalado de forma global mediante el comando "**npm install -g @ionic/cli**".
4. **Navegador Web:** Google Chrome, Edge o Firefox.

Instalación de dependencias

Para descargar e instalar todos los módulos necesarios definidos en el archivo `package.json`, ejecute en la terminal dentro de la raíz del proyecto:

npm install

Ejecución local

Para iniciar el servidor de desarrollo local y previsualizar la aplicación en el navegador:

ionic serve

Construcción del proyecto

Para compilar y generar los archivos estáticos de producción (ubicados en la carpeta `www/` o `dist/`):

ionic build

Ejecución exitosa del comando **ionic serve** en la terminal

```
PS C:\Users\victo\OneDrive\Desktop\ProyectoFinalNacho\proyectonacho> ionic serve
> ng.cmd run app:serve --host=localhost --port=8100
[ng] > Building...
[ng] ✓ Building...
[ng] Initial chunk files | Names | Raw size
[ng] styles.css | styles | 57.36 kB
[ng] main.js | main | 3.06 kB
[ng] chunk-Y5RQAIA6.js | - | 1.50 kB
[ng] polyfills.js | polyfills | 143 bytes
[ng] | Initial total | 62.05 kB
[ng] Lazy chunk files | Names | Raw size
[ng] tab1.page-J2FX5IQV.js | tab1-page | 34.53 kB
[ng] tab3.page-XJXE2LD8.js | tab3-page | 19.83 kB
[ng] tab2.page-H7K7MKB2.js | tab2-page | 17.18 kB
[ng] Application bundle generation complete. [6.064 seconds] - 2026-07-28T03:39:38.596Z
[ng] NOTE: Raw file sizes do not reflect development server per-request transformations.

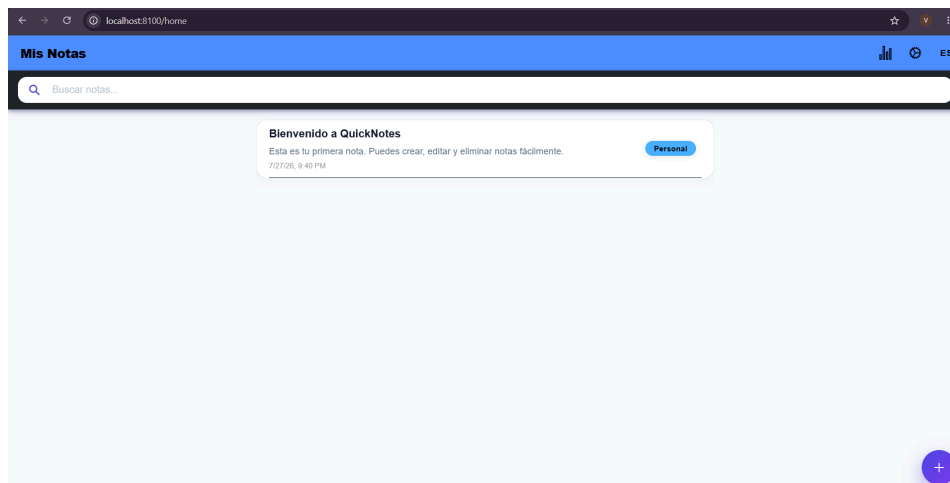
[INFO] Development server running!

Local: http://localhost:8100

Use Ctrl+C to quit this process

[ng] → Local: http://localhost:8100/
[ng] → press h + enter to show help
[INFO] Browser window opened to http://localhost:8100!
```

Aplicación ejecutándose en la ventana del navegador local

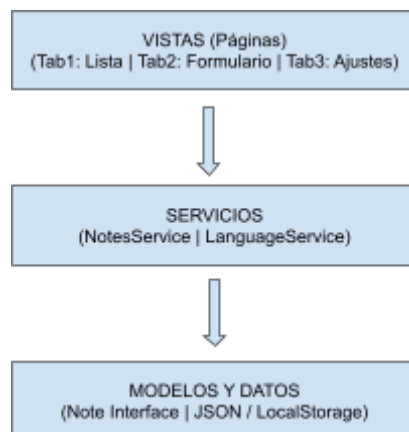


1.3 Arquitectura de la aplicación

La arquitectura del proyecto sigue el patrón de diseño modular de Angular/Ionic, aplicando una clara separación de responsabilidades:

- **Páginas (src/app/tab1, tab2, tab3, tabs):** Representan las vistas principales de la interfaz gráfica.
 - **Tab1:** Vista principal encargada de desplegar la lista de notas.
 - **Tab2:** Formulario dedicado al registro y edición de notas.
 - **Tab3:** Panel de configuración para la selección de idioma y ajustes generales.
 - **Tabs:** Layout contenedor que maneja la barra de navegación por pestañas.
- **Componentes (src/app/explore-container):** Elementos visuales reutilizables diseñados para encapsular interfaz y comportamiento específico.
- **Servicios (src/app/services):**
 - **NotesService:** Administra la lógica de negocio, el almacenamiento y las operaciones CRUD sobre las notas.
 - **LanguageService:** Gestiona el estado global del idioma preferido por el usuario.
- **Modelos o Interfaces (src/app/models/note.model.ts):** Define la estructura de datos fuertemente tipada de la entidad Note (id, title, content, date, category).
- **Archivos de traducción (src/assets/i18n/):** Contienen los diccionarios de claves en formato JSON (es.json y en.json).

Diagrama de Arquitectura



Justificación de Mantenimiento

La separación de responsabilidades facilita el mantenimiento del software, ya que cualquier cambio en la interfaz gráfica no altera la lógica del negocio ni el modelo de datos[cite: 1]. Por ejemplo, si se decide cambiar la forma en que se almacenan las notas (de `LocalStorage` a una API REST externa), solo es necesario modificar el servicio `NotesService` sin afectar las páginas ni los componentes[cite: 1, 2].

1.4 Internacionalización

Biblioteca y Mecanismo

La internacionalización se implementó mediante el servicio `LanguageService`, el cual gestiona la carga de diccionarios JSON locales y expone la variable de idioma activa para toda la aplicación[cite: 1, 2].

Estructura de Archivos de Traducción

Los archivos de traducción se encuentran alojados en la ruta `src/assets/i18n/`[cite: 1, 2]: * `es.json` (Diccionario en español)[cite: 1, 2] * `en.json` (Diccionario en inglés)[cite: 1, 2]

Elementos Traducidos

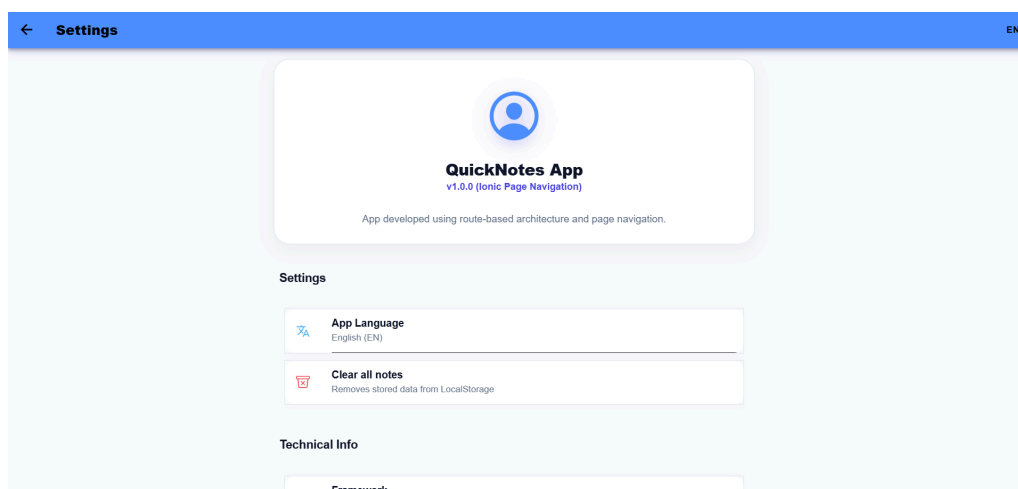
Se parametrizaron con claves de traducción los siguientes elementos[cite: 1]:

- Títulos y encabezados de la aplicación[cite: 1].
- Menús y etiquetas de pestañas de navegación[cite: 1].
- Botones de acción (Guardar, Cancelar, Eliminar)[cite: 1].
- Etiquetas de los formularios e instrucciones[cite: 1].
- Mensajes de validación y alertas de información/error[cite: 1].

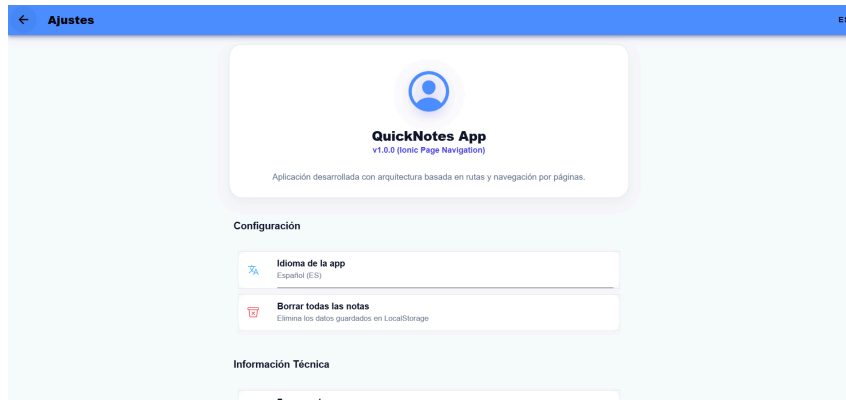
Cambio de Idioma

El usuario puede alternar el idioma de la aplicación en cualquier momento desde la pestaña de configuración (`Tab3`)[cite: 1, 2]. Al presionar el botón de selección, se invoca la función de cambio en `LanguageService`, actualizando dinámicamente las cadenas de texto sin necesidad de reiniciar la aplicación[cite: 1, 2].

Pantalla en Ingles



Pantalla en Español



6.5 Control de versiones[cite: 1]

El equipo utilizó **Git** como sistema de control de versiones distribuido y **GitHub** como plataforma de hospedaje remoto[cite: 1].

Creación del Repositorio y Flujo

1. Se inicializó el repositorio local con ``git init`` y se vinculó al servidor remoto en GitHub[cite: 1].
2. Se estableció la rama ``main`` como la rama principal y estable[cite: 1].
3. Para mantener el orden, cada integrante trabajó en ramas específicas de funcionalidad[cite: 1].

Estructura de Ramas Utilizadas

- ``main``: Rama de código estable lista para construcción y despliegue[cite: 1].
- ``feature/notes-service``: Rama dedicada a la lógica de notas y modelos[cite: 1, 2].
- ``feature/i18n-support``: Rama para la configuración de traducciones y cambio de idioma[cite: 1, 2].

Registro de Cambios e Integración

Los cambios se registraron de manera incremental utilizando commits con mensajes descriptivos[cite: 1]. La integración hacia la rama ``main`` se realizó mediante fusiones (``git merge``) tras verificar que el código compilara correctamente[cite: 1].

Ejemplos de Commits

- ``git commit -m "Agrega modelo Note y servicio NotesService"``
- ``git commit -m "Implementa formulario de notas con validaciones"``
- ``git commit -m "Configura soporte bilingue en es.json y en.json"``

Integración continua con despliegue automatizado

Para la automatización del proyecto, el equipo configuró un flujo básico mediante GitHub Actions. Este proceso se ejecuta automáticamente cada vez que se detecta un push en la rama “main” y está definido por el siguiente archivo de configuración:

```
name: Build and Deploy Ionic App

on:
  push:
    branches:
      - main

permissions:
  contents: write

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout del código
        uses: actions/checkout@v4

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Instalar dependencias
        run: npm ci

      - name: Compilar la aplicación Ionic
```

```
run: npx ng build --configuration production --base-href /proyectonacho/  
--output-path www
```

```
- name: Desplegar en GitHub Pages  
  
uses: JamesIves/github-pages-deploy-action@v4  
  
with:  
  
  folder: www/browser  
  
  branch: gh-pages
```

Explicación de cada paso:

1. **Descarga del código:** Se utiliza la acción checkout@v4 para que el entorno virtual descargue la última versión del repositorio.
2. **Instalación de dependencias:** Primero se configura el entorno con Node.js en su versión 20 y posteriormente se ejecuta “npm ci” para realizar una instalación limpia y exacta de las dependencias.
3. **Construcción de la aplicación:** Se ejecuta el comando de compilación de Angular/Ionic (npx ng build), indicando que es para producción y asignando la ruta base “/proyectonacho/” para que los archivos estáticos carguen correctamente. El resultado se guarda en la carpeta “www”.
4. **Despliegue automatizado:** Se utiliza una acción externa que toma los archivos compilados en la ruta “www/browser” y los sube automáticamente a la rama “gh-pages” para publicarlos.

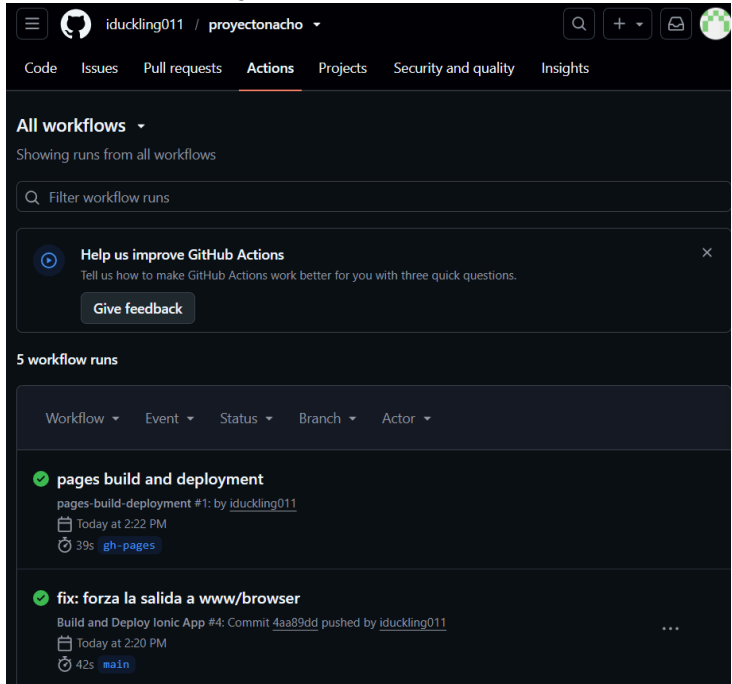
Manejo de fallos:

Si la construcción falla en cualquiera de estas etapas (por ejemplo, por un error de sintaxis que impida compilar con “ng build” o un fallo en la instalación de dependencias), el flujo de GitHub Actions se detiene inmediatamente. El código roto no llega a la rama “gh-pages”, por lo que la versión en línea no se ve afectada, y la plataforma nos alerta del error para poder revisar los registros y corregirlo.

Tras una construcción exitosa, la aplicación se publica en GitHub Pages.

- **Enlace público de la aplicación:** <https://iduckling011.github.io/proyectonacho/>

- **Captura de una ejecución correcta:**



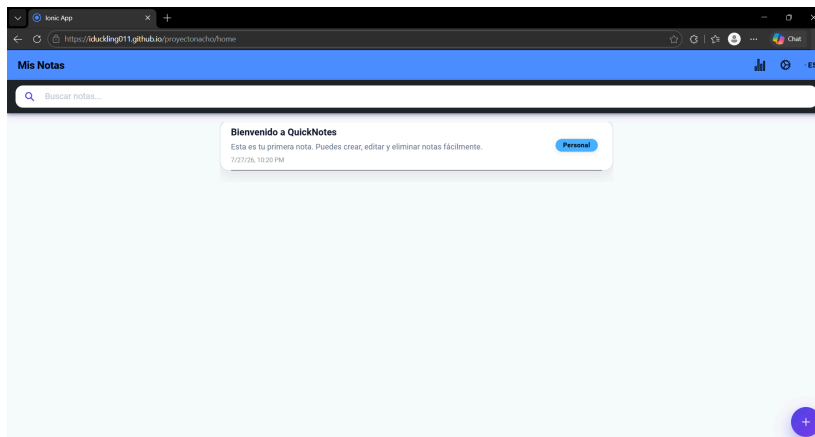
Pruebas realizadas

Prueba	Resultado esperado	Resultado obtenido	Estado
Navegación	Las páginas abren correctamente	Al interactuar con el menú, las vistas cambian de forma fluida entre la lista de notas y la vista para agregar una nueva, sin recargar la página entera.	Correcta
Formulario	Valida campos obligatorios	El botón de "Guardar" se mantiene deshabilitado hasta que el usuario ingresa un título y contenido válido para la nota.	Correcta
Cambio de idioma	La interfaz cambia de idioma	Al seleccionar inglés o español, los títulos, botones y mensajes de error se actualizan inmediatamente usando los archivos de traducción.	Correcta
Construcción	El proyecto compila correctamente	El comando de compilación terminó sin arrojar errores de dependencias en el flujo de GitHub Actions.	Correcta
Despliegue	La aplicación abre desde el enlace público	La aplicación es accesible y funcional al ingresar desde el navegador web mediante el enlace generado por GitHub Pages.	Correcta

Problemas, causas, soluciones

Caso 1: Pantalla en blanco al desplegar en GitHub Pages

- **Problema encontrado:** Una vez que GitHub Actions reportó la publicación exitosa, al entrar al enlace público de GitHub Pages la pantalla se quedaba en blanco y no cargaba la lista de notas.
- **Causa probable:** Ionic compila la app apuntando los recursos a la raíz del servidor, pero GitHub Pages sirve la app en una subcarpeta con el nombre del repositorio.
- **Solución aplicada:** Se ajustó el script de compilación en el archivo ".yml" de GitHub Actions para agregar el flag del base href: "npm run build -- --base-href=/proyectonacho".
- **Evidencia del resultado:**



Caso 2: Mensajes de error anticipados en el formulario de notas

- **Problema encontrado:** El mensaje de advertencia de "El título es obligatorio" aparecía inmediatamente al entrar a la página para crear una nota, antes de que el usuario intentara escribir algo.
- **Causa probable:** La validación del formulario en el HTML únicamente estaba comprobando si el input era "inválido", omitiendo verificar si el usuario ya había interactuado con él.
- **Solución aplicada:** Se agregó la condición de que el input debía haber sido tocado para poder mostrar el mensaje de advertencia rojo.
- **Evidencia del resultado:**

```
<!-- Modal para Agregar / Editar Nota -->
<ion-modal [isOpen]="isModalOpen" (didDismiss)="closeModal()">
  <ng-template>
    <ion-header>
      <ion-toolbar color="primary">
        <ion-title>
          {{ currentNote.id ? (lang === 'es' ? 'Editar Nota' : 'Edit Note') : (lang === 'es' ? 'Nueva Nota' : 'New Note') }}
        </ion-title>
        <ion-buttons slot="end">
          <ion-button (click)="closeModal()">
            <ion-icon slot="icon-only" name="close-outline"></ion-icon>
          </ion-button>
        </ion-buttons>
      </ion-toolbar>
    </ion-header>

    <ion-content class="ion-padding">
      <ion-item class="ion-margin-bottom">
        <ion-label position="stacked">{{ lang === 'es' ? 'Título *' : 'Title *' }}</ion-label>
        <ion-input [(ngModel)]="currentNote.title" [placeholder]="lang === 'es' ? 'Escribe el título' : 'Enter title'"></ion-input>
      </ion-item>

      <ion-item class="ion-margin-bottom">
        <ion-label position="stacked">{{ lang === 'es' ? 'Categoría' : 'Category' }}</ion-label>
        <ion-select [(ngModel)]="currentNote.category" [placeholder]="lang === 'es' ? 'Selecciona una' : 'Select one'">
          <ion-select-option value="Personal">{{ lang === 'es' ? 'Personal' : 'Personal' }}</ion-select-option>
          <ion-select-option value="Trabajo">{{ lang === 'es' ? 'Trabajo' : 'Work' }}</ion-select-option>
          <ion-select-option value="Estudios">{{ lang === 'es' ? 'Estudios' : 'Studies' }}</ion-select-option>
        </ion-select>
      </ion-item>
    </ion-content>
  </ng-template>
</ion-modal>
```

Conclusión

Ian Durante este proyecto aprendí a integrar diferentes herramientas para crear una aplicación funcional. En el equipo, me encargué de desarrollar la estructura visual de las vistas principales y el formulario de notas, además de apoyar en la redacción de este manual técnico. Mi principal dificultad fue organizar correctamente los componentes de Ionic para mantener un buen diseño, así como resolver algunos conflictos al momento de unir el código en Git. Considero que la mayor ventaja de automatizar el despliegue es el ahorro de tiempo, ya que nos permite enfocarnos directamente en mejorar el código sin preocuparnos por subir archivos manualmente.

Victor En esta práctica aprendí a implementar la internacionalización en una aplicación para que funcione en dos idiomas. Mi participación consistió en configurar los archivos de traducción en JSON, programar la lógica básica para guardar las notas y colaborar en la creación de este documento. La principal dificultad que encontré fue lograr que los mensajes de error en los formularios cambiaran de idioma correctamente sin afectar el funcionamiento de la pantalla. La mejor ventaja de la automatización es tener una versión del proyecto siempre disponible en línea, lo cual nos facilitó mucho hacer pruebas desde nuestros teléfonos móviles.

Arturo Con el desarrollo de esta aplicación aprendí cómo funciona GitHub Actions y la utilidad de aplicar DevOps en un proyecto escolar. Me enfoqué en configurar la automatización del despliegue, solucionar la publicación en GitHub Pages y estructurar la información de este manual junto con mis compañeros. Mi principal dificultad fue identificar el problema con las rutas al subir la página, el cual se solucionó ajustando el parámetro “--base-href” en la compilación. La gran ventaja de este proceso automatizado es la seguridad que ofrece: si el código tiene algún error, el sistema detiene la publicación y evita que la aplicación final deje de funcionar.